SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 - Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 - Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills		
Student Name:	**Fermy Selvanathan**	**Reg. No.:**	**1921260299**
Date:	**27/10/26**	**Duration:**	60 minutes

Code of Conduct

I Fermy Selvanathan - 1921260299 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *Fermy Selvanathan*

Annexure A - Architecture Design Assignment

Based on your assigned problem statement, perform an Architecture Design Analysis and prepare an architecture design report by answering the following questions:

Architecture Design of "CareConnect" - A Cloud-Based Telemedicine and Electronic Health Record Platform for Hospitals and Clinics

Problem Statement: Hospitals and clinics need a secure, scalable and highly available platform that lets patients book appointments and consult doctors by video, lets doctors access a patient's complete health record and issue eprescriptions, and exchanges data with labs, pharmacies, insurers and national health networks - while protecting sensitive medical data and recording patient consent.

1. Analyze System Requirements

1.1 Study the problem statement and identify the system objectives

CareConnect connects patients, doctors, nurses, pharmacists and lab staff across many hospitals and clinics. It is a **privacy-critical, real-time, interoperability-heavy** platform: medical records are among the most sensitive personal data, video consultations must start quickly and stay stable on weak mobile networks, and data must flow to external systems (labs, insurers, national health networks) in standard formats. Downtime or a wrong record can directly affect patient safety.

1. **Appointments:** search doctors by specialty, view live slots, book, reschedule and cancel with automatic reminders.
2. **Teleconsultation:** secure video / audio / chat consultation between patient and doctor with a reliable fallback on poor networks.
3. **Electronic Health Record (EHR):** maintain a unified longitudinal record (history, diagnoses, vitals, lab reports, scans) using standard FHIR resources.
4. **Prescriptions and diagnostics:** issue e-prescriptions with drug-interaction checks; order lab tests and receive results.
5. **Consent and privacy:** patients decide who can see their records; every access is logged.
6. **Billing and claims:** consultation fees, online payment, invoices and insurance claim submission.
7. **Interoperability and extensibility:** exchange data with national health networks (ABDM), lab and pharmacy systems; add new modules such as remote patient monitoring later.

1.2 Functional requirements that influence the architecture

ID	Functional Requirement	Architectural Influence
FR1	Register users, login by OTP / SSO, role-based access (patient, doctor, nurse, pharmacist, lab, admin)	Identity & Consent service, OIDC identity provider, RBAC, JWT at the gateway
FR2	Doctor search, slot management, booking, rescheduling, reminders	Appointment & Scheduling service; cached slot availability; asynchronous reminder worker
FR3	Real-time video / audio / chat consultation with screen and report sharing	WebRTC with signalling gateway and media server (SFU / TURN); Teleconsultation Session service
FR4	Create, read and update patient records; attach scans and reports; full history view	EHR service built on FHIR resources; object storage for large files; readoptimised search index
FR5	Issue e-prescriptions with drug and allergy checks; send to pharmacy	Prescription & Pharmacy service; rule / interaction checks; pharmacy adapter
FR6	Order lab tests, receive and attach results	Lab & Diagnostics service with HL7 / FHIR integration
FR7	Billing, online payment, invoices and insurance claims	Billing & Claims service; payment adapter (tokenised); asynchronous claims worker
FR8	Consent management, access history for patients, audit reports for administrators	Consent store, immutable Audit & Access Log, event-based record.accessed logging

1.3 Non-functional requirements that influence the architecture

Category	Measurable Requirement	Architectural Influence
Performance	Appointment search / booking p95 < 500 ms; patient summary loads within 2 s; video consultation joins within 5 s; end-to-end video latency under 300 ms	Redis cache, indexed search, media server close to users, adaptive bitrate
Scalability	200 hospitals / clinics, 2,000 doctors, 500,000 patients; 5,000 concurrent portal users and 1,000 concurrent video consultations at peak; 10x growth without redesign	Stateless services, horizontally scalable media servers, partitioned event stream
Availability	99.95% for booking and records; consultations must survive failure of one media node	Multi-AZ deployment, replicated databases, automatic media failover
Reliability / Durability	No loss of clinical records or prescriptions; RPO <= 1 minute for records, RTO <= 30 minutes; reconnect and resume dropped consultations	Synchronous replication for EHR, backups, session resume, idempotent APIs
Security and Privacy	Encryption in transit (TLS 1.3) and at rest (AES-256); MFA for staff; consent-based access; compliance with the DPDP Act 2023, telemedicine guidelines and ABDM norms; masking of sensitive fields	Gateway + WAF, field-level encryption, consent enforcement point, secrets vault

Interoperability	Support HL7 FHIR R4, REST, OIDC / SAML and standard lab messages	FHIR-native EHR, integration adapters behind stable internal contracts
Maintainability / Auditability	New module within one sprint; every record access (including emergency "break-glass" access) logged immutably and retained per policy	Independent services, CI/CD, append-only audit store

1.4 Key quality attributes (ranked by influence on the architecture)

Rank	Quality Attribute	Why it dominates in CareConnect
1	**Security & Privacy**	Medical data is highly sensitive and regulated; a breach harms patients and violates law.
2	**Availability & Reliability**	Clinicians depend on records during treatment; lost records or failed consultations affect patient safety.
3	**Interoperability**	Value of the system depends on exchanging data with labs, pharmacies, insurers and national networks in standard formats.
4	**Scalability & Performance**	Peak-hour booking traffic and real-time video need elastic capacity and low latency.
5	**Maintainability & Auditability**	Regulations, standards and clinical workflows change; every access to a record must be provable.

2. Select a Suitable Software Architecture

2.1 Selected architectural style

Hybrid Architecture = Layered Client-Server + Service-Oriented (API-first) Microservices + Event-Driven Processing + a dedicated Real-Time Media Tier. Web and mobile clients call domain services through an API gateway; each domain (Appointments, EHR, Teleconsultation, Prescriptions, Lab, Billing, Identity & Consent) is a separate service with its own data store and a versioned, FHIR-aligned API. Real-time video runs on a separate media tier (WebRTC signalling gateway, SFU and TURN servers) so that heavy media traffic never competes with business APIs. Non-urgent work (reminders, claims, indexing, notifications) is handled asynchronously through Apache Kafka. A central consent and audit layer controls and records every access to health data. The platform runs in containers (Kubernetes) across multiple availability zones.

2.2 Evaluation of candidate architectural styles

Style	Scalability (incl. real-time media)	Security / data isolation	Interoperability / evolution	Complexity / cost	Verdict
Single layered monolith (MVC)	**Poor - video and booking scale together**	**Weak isolation of sensitive data**	**Low**	**Low**	Rejected: cannot isolate video load or sensitive modules
Client-Server only (2tier)	**Poor**	**Poor**	**Low**	**Low**	Rejected: no support for scale, integration or consent control
Serverless (FaaS) only	**Good for APIs, unsuited to long video sessions**	**Good**	**Good**	**Cold starts, limits, lockin**	Partly usable for small jobs, not for media

Classic SOA with ESB	**Medium**	**Medium**	**Good for HL7 but heavy**	**Heavy central middleware**	Rejected: ESB bottleneck and cost
Pure event-driven only	**Excellent**	**Good**	**Good**	**Hard for requestresponse flows (booking, login, record view)**	Used only where asynchrony fits
Hybrid: Layered + Services (API-first) + Event-Driven + Media Tier	**Excellent**	**Excellent per-service data, central consent**	**Excellent - FHIR adapters**	**Moderate (managed Kubernetes / Kafka)**	**Selected**

2.3 Why the selected architecture is the most suitable

- **Real-time video must be isolated.** Media streaming is bandwidth- and CPU-heavy; a separate tier lets it scale on its own and fail without affecting records or booking.
- **Sensitive data needs boundaries.** Service-per-domain with database-per-service limits exposure, and a central consent / audit layer enforces who may see what.
- **Interoperability by design.** API-first, FHIR-aligned services and adapters make exchange with ABDM, labs and insurers a configuration task, not a rewrite.
- **Asynchronous work does not block clinicians.** Reminders, claims and indexing run through Kafka workers and can be retried safely.
- **Fault isolation and availability.** A failure in billing or notification does not stop a consultation or access to records. **Evolution.**
- New services (remote monitoring, AI triage) can subscribe to events without changing existing ones.

3. Draw the Software Architecture Diagram

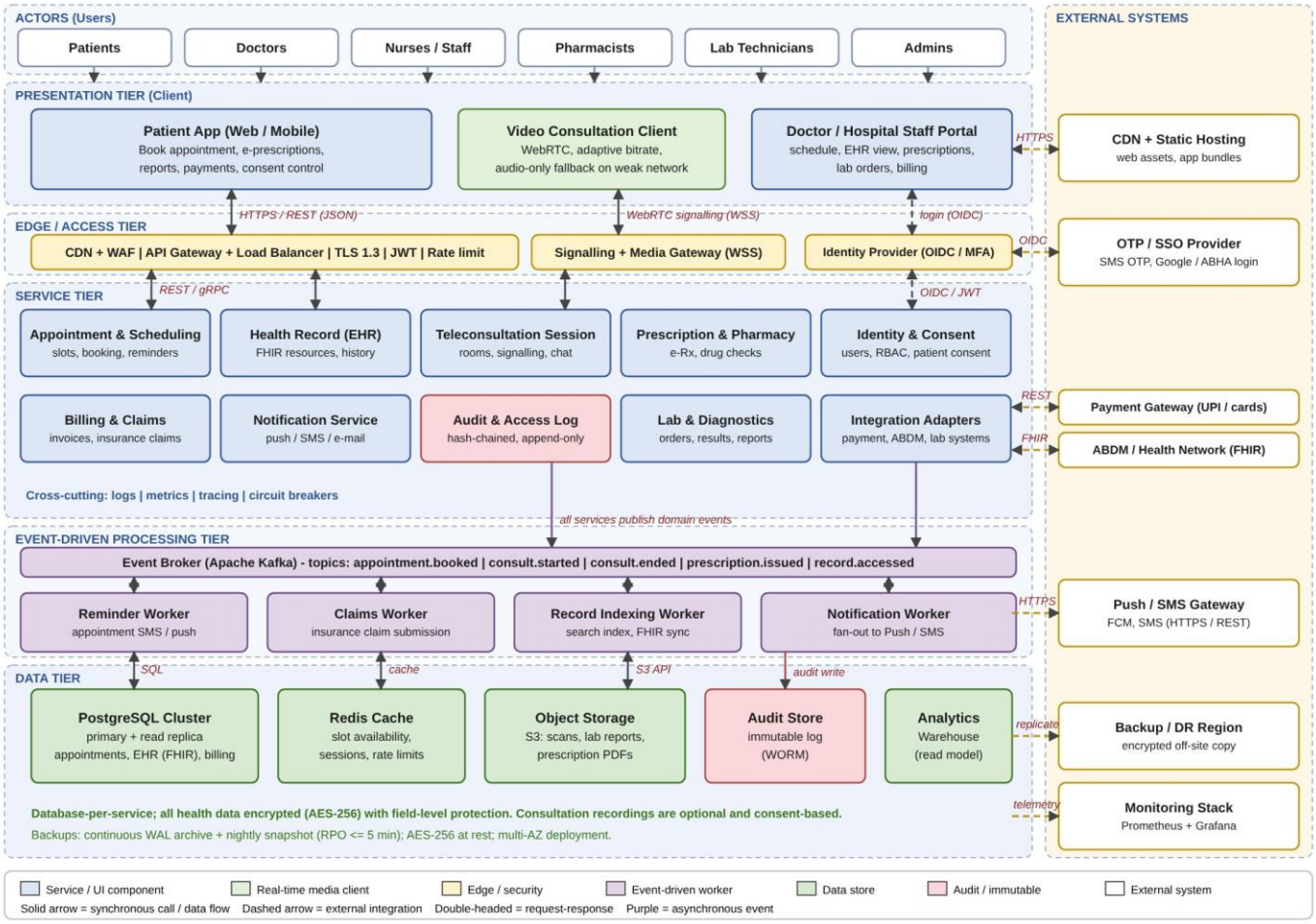


Figure 1: Layered, service-oriented and event-driven architecture of the CareConnect platform

3.1 Notation and reading guide

- **Tiers (dashed bands):** Actors, Presentation, Edge/Access, Service, Event-Driven Processing and Data tiers; external systems are shown in the right-hand column.
- **Boxes:** blue = business services / UI, green (top) = real-time media client, yellow = security and edge components, purple = event broker and asynchronous workers, green (bottom) = data stores, red = audit / immutable components, white = external systems.
- **Arrows:** solid double-headed = synchronous request-response (HTTPS / REST / gRPC / WebRTC signalling over WSS / SQL); purple = asynchronous domain events on Kafka; dashed = external integrations (OIDC, REST, FHIR, HTTPS, replication, telemetry). Labels on the arrows state the protocol or data carried.

4. Explain Components and Their Interactions

4.1 Purpose and responsibilities of each component

Component	Responsibility	Technology (proposed)
Patient App (Web / Mobile)	Doctor search, booking, e-prescriptions, reports, payments and consent controls	React / Flutter, push notifications
Video Consultation Client	Captures camera / microphone, adapts bitrate, falls back to audio-only on weak networks, supports chat and file sharing	WebRTC, adaptive streaming
Doctor / Hospital Staff Portal	Schedules, patient lists, EHR view and update, prescriptions, lab orders, billing screens	React SPA

CDN + WAF + API Gateway + Load Balancer	Serves static assets, terminates TLS, filters attacks, validates JWT, applies rate limits, routes REST traffic	NGINX / Kong, cloud WAF
Signalling + Media Gateway	WebSocket signalling for call set-up; routes media through SFU / TURN servers; scales independently	WSS, SFU (mediasoup / Janus), coturn
Identity Provider + Identity & Consent Service	OTP / SSO login, MFA for staff, tokens, roles; stores patient consent and enforces it on every data request	Keycloak (OIDC), policy engine
Appointment & Scheduling Service	Doctor calendars, slot locking, booking and rescheduling rules	Spring Boot, PostgreSQL, Redis
Health Record (EHR) Service	Stores and serves FHIR resources (Patient, Encounter, Observation, Condition, DiagnosticReport); versioned history	HAPI FHIR, PostgreSQL, object storage
Teleconsultation Session Service	Creates consultation rooms, issues short-lived media tokens, tracks session state and notes	Node.js / Spring Boot, Redis
Prescription & Pharmacy Service	Creates e-prescriptions, checks drug interactions and allergies, forwards to pharmacies	Spring Boot, rules engine
Lab & Diagnostics Service	Test orders, result intake, report attachment to the EHR	HL7 / FHIR integration
Billing & Claims / Notification / Integration Adapters	Invoices and insurance claims; push / SMS / e-mail; wrappers for payment, ABDM and external lab systems	REST adapters, FCM, SMS API
Audit & Access Log Service and Store	Records every record access, consent change and prescription in an append-only, hash-chained log; supports break-glass review	WORM store / append-only table
Event Broker + Workers	Kafka decouples services; Reminder, Claims, Record Indexing and Notification workers scale with queue lag	Apache Kafka, containerised consumers
Data stores	PostgreSQL (primary + replica), Redis (cache), object storage (scans, reports, PDFs), analytics warehouse (de-identified), audit store (immutable)	PostgreSQL, Redis, S3

4.2 How components communicate and exchange data

From - To	Mechanism	Data exchanged
App / portal - API Gateway	HTTPS / REST (JSON), TLS 1.3	Login, search, booking, record access, prescriptions with JWT
Video client - Signalling Gateway	WebRTC signalling over WSS; media over SRTP via SFU / TURN	Offer / answer, ICE candidates, encrypted audio-video streams
Gateway - Services	REST / gRPC with service discovery	Authenticated requests with user claims and consent token
Services - Identity & Consent	OIDC / JWT validation; policy check	Tokens, roles, consent decisions

Services - Event Broker	Kafka publish / subscribe	Events: appointment.booked, consult.started, consult.ended, prescription.issued, record.accessed
Workers - Data stores	SQL, cache protocol, S3 API	Claims status, search index updates, reports
Adapters - Payment gateway / ABDM / labs	REST over HTTPS, FHIR / HL7	Payment tokens; health-record exchange; test orders and results
Notification Worker - Push / SMS gateway	HTTPS	Reminders, consultation links, prescription alerts
All services - Audit Service	Async events	Who accessed what, when, why; hash of previous record

4.3 Overall workflow of the system

Login & consent -> **Search & book appointment** -> **Reminder** -> **Video consultation** -> **EHR update & e-prescription** -> **Lab / pharmacy** -> **Billing & claim** -> **Follow-up**

1. **Login:** The patient signs in with OTP / SSO; the identity provider issues a JWT, validated on every call at the gateway.
2. **Search and book:** The Appointment service reads slot availability from Redis, locks the slot, creates the appointment and publishes appointment.booked.
3. **Reminder and payment:** The Reminder worker sends SMS / push alerts; Billing collects the fee through the payment adapter.
4. **Join consultation:** At the scheduled time both parties join; the Teleconsultation service issues media tokens, the signalling gateway sets up the call and the SFU relays encrypted media. On a weak network the client drops to audioonly.
5. **Clinical work:** With the patient's consent the doctor views the EHR (FHIR resources from the EHR service), records notes and diagnosis, and shares reports on screen. Each access publishes record.accessed to the audit log.
6. **Prescribe and order:** The doctor issues an e-prescription (interaction and allergy checks) and lab orders; prescription.issued is sent to the pharmacy and lab through adapters.
7. **Results:** Lab results arrive in FHIR / HL7 format and are attached to the patient record; the patient and doctor are notified.
8. **Close:** consult.ended triggers invoice generation and the insurance claim through the Claims worker; the search index is updated; all actions are in the audit store; follow-up reminders are scheduled.

5. Discuss Quality Attributes

Attribute	How the architecture addresses it (tactics)	Target / evidence
Scalability	Stateless services scale horizontally with Kubernetes HPA; SFU media servers scale by concurrent rooms with consistent roomto-node routing; Kafka partitions and parallel consumers; Redis absorbs read-heavy slot lookups; read replicas and a separate de-identified analytics store; CDN for static content; hospitals / tenants partitioned by tenant ID.	1,000 concurrent consultations now; 10x by adding media nodes and partitions
Security	TLS 1.3 everywhere, SRTP for media; AES-256 encryption at rest with field-level encryption for sensitive data; OIDC with MFA for staff; RBAC plus patient-consent enforcement on every request; WAF, rate limiting and DDoS protection; secrets vault; hash-chained audit log; break-glass access with mandatory justification and review; de-identification before analytics; consultation recording only with consent.	Consent enforced at each data access; no unlogged access; DPDP Act 2023 aligned
Performance	Cached slot availability, indexed search, API pagination; preaggregated patient summaries; media servers deployed near users with adaptive bitrate; asynchronous claims, indexing and notifications; connection pooling.	Booking p95 < 500 ms; patient summary < 2 s; call join < 5 s

Reliability	Synchronous replication and point-in-time recovery for the EHR; idempotent booking and prescription APIs; session resume after network drop; Kafka retention for replay; circuit breakers and retries; automatic media-node failover; reconciliation jobs for payments and claims.	RPO <= 1 min for records; RTO <= 30 min
Maintainability	Domain-aligned services with small codebases; versioned FHIRaligned APIs and event schemas; adapters isolate external systems; independent CI/CD with canary releases; configurationdriven clinical rules; centralised logs, metrics and tracing; infrastructure as code.	New module or integration without touching unrelated services
Availability	Multi-AZ deployment, at least two replicas per service, health checks and auto-restart, managed PostgreSQL failover, replicated Kafka and Redis, DR region with backups, blue-green releases without downtime, read-only emergency mode for records if writes are degraded, 24x7 monitoring.	99.95% availability; no single point of failure

6. Justify Architectural Decisions

6.1 Rationale behind the architectural choices

ID	Decision	Rationale	Alternative considered
D1	Separate real-time media tier (signalling + SFU / TURN)	Heavy streaming traffic isolated from business APIs; scales and fails independently	Routing media through application servers (overload)
D2	Domain microservices with database-per-service	Limits exposure of sensitive data; independent scaling and releases	Modular monolith (simpler, weaker isolation)
D3	FHIR-based EHR and API-first integration	Standard format for ABDM, labs and insurers; reduces custom mapping	Proprietary record schema (costly integration)
D4	Central consent and audit enforcement	Meets privacy law and builds patient trust; evidence for audits and disputes	Consent checks inside each service (inconsistent)
D5	Kafka event backbone for reminders, claims, indexing, notifications	Decouples slow tasks from clinicians; durable, replayable, retry-safe	Direct calls (cascading failures)
D6	External identity provider (OIDC) with MFA	Centralised authentication; reuse of hospital / national identities	Custom authentication in each service
D7	Adapters for payment, ABDM and lab systems	Isolates the core from third-party changes; tokenisation keeps card data out of scope	Direct integrations from each service
D8	Kubernetes, multi-AZ deployment	Auto-scaling, self-healing, rolling updates, cloud portability	Fixed VMs (manual scaling and recovery)

6.2 Trade-offs considered

Trade-off	Cost accepted	Mitigation
Microservices vs. simplicity	More operational complexity and network hops; harder debugging	About 8 coarse-grained services; managed Kubernetes / Kafka; distributed tracing; service templates
Security vs. usability	MFA, consent prompts and access justification add friction for doctors and patients	Risk-based MFA, remembered devices, onetime consent per episode of care, breakglass for emergencies

Strong vs. eventual consistency	Search index, analytics and claims lag live data by seconds	Strong consistency for records, prescriptions and slot booking; eventual consistency only for derived data
Own media servers vs. managed video service	Self-hosting needs expertise; a managed service costs more and shares data with a third party	Self-hosted SFU for data control, with a managed provider as backup option behind the same signalling API
Cost vs. availability	Multi-AZ replicas and DR region increase cost	Auto-scale non-critical services; right-size replicas; reserve capacity for the core clinical path

6.3 Support for future enhancements, integration and long-term maintainability

- **New modules as new consumers:** remote patient monitoring (wearables), AI-based triage or chronic-care programmes can subscribe to existing Kafka events and call FHIR APIs without modifying current services.
- **National and partner integration:** FHIR-native APIs and adapters make it straightforward to connect additional health networks, insurers, pharmacies and labs.
- **Multi-hospital growth:** tenant IDs in tokens and data partitioning allow new hospitals to join one platform; regional deployments can be added by replicating the stack.
- **Replaceable components:** media servers, identity provider and payment gateway sit behind stable interfaces and can be swapped without affecting clinical services.
- **Regulatory change:** consent rules, retention periods and audit reports are configuration-driven and centralised, so new regulations affect one layer, not every service.
- **Operational maintainability:** CI/CD, infrastructure as code, standard logs / metrics / traces, runbooks and architecture decision records keep the system understandable as teams change.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and nonfunctional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.

Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of tradeoffs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	**/25**